# Generating Company Names with a Character-Level Transformer

Mauro Reverberi

Udacity Institute of AI & Technology / Woolf

Master of Science in AI, Capstone Project 5: Generative AI Applications

November 3, 2026

# Generating Company Names with a Character-Level Transformer

## Overview

In this project I built a small generative model that writes new UK company names, one character at a time. The model is a decoder-only Transformer that I built with the GPT-2 classes of the Hugging Face Transformers library (Wolf et al., 2020) and trained from scratch in PyTorch, without any pretrained weights. It learned from 80,142 names of real private limited companies on the Companies House register. My research question was whether such a model can generate plausible and varied company names. To answer it I counted repeated names to measure variety, read a random sample to judge plausibility, and checked how often a generated name copied a training name or had the same key as a company in the register. The task connects to my earlier capstone projects, which all work with company data for compliance checks. A later project will build a due diligence agent that matches company names against registers, and generated names could serve as extra test inputs for that step. On their own they are not a matching benchmark, which would need pairs of names that are known to be the same or different companies.

A language model fits this task because a company name is a short sequence of characters, and generating it means predicting one character after the other. Of the three options in the project (GAN, VAE and Transformer), I chose the Transformer because self-attention lets every position use all earlier characters of the sequence directly (Vaswani et al., 2017). The decoder-only Transformer from my coursework is trained for exactly this kind of next-character prediction and samples one character after the other, which fits short names well. A GAN or a VAE would need a second network, a discriminator or an encoder, and the GAN also a way to

train through discrete characters (Goodfellow et al., 2016). The model reached a test loss of 1.1513 nats per character (the cross-entropy computed with the natural logarithm, lower is better) against 1.3819 for a character 5-gram model. Almost all generated names end correctly, and 16 of 8,000 are exact copies of training names. The most important finding is that at temperature 0.5 the key of 44.25% of the names (885 of 2,000) matches a company in the snapshot under my own register key. Across all four temperatures there were 1,618 such matches. These are possible collisions with real companies, and only 16 of them are exact copies of training names.

## Dataset or Prompt Description

Companies House publishes a free monthly snapshot of all live companies on the UK register, with name, company number, address, category, status and other fields (Companies House, n.d.). I downloaded the snapshot of October 1, 2026 on October 16, 2026, a ZIP file of 494 MB with one CSV file of 2.8 GB and 5,704,711 companies. Companies House does not limit how the information on the public register may be used, but possible rights of third parties in the data still apply (Companies House, 2019). The names are real register entries, nothing in the data is synthetic or generated, and I did not use this source in my earlier capstone projects.

My script prepare_dataset.py reads the snapshot once. It keeps the 5,276,662 private limited companies, the largest category, so all training names share one legal form and one set of endings. It then removes names with characters outside an allowed set of capital letters, digits and common punctuation (6,984), for example names with accented letters or quotation marks. It also removes names longer than 60 characters (2,415) and names that do not end with LTD, LTD. or LIMITED (7,971). From the remaining 5,259,292 names it removes 11 that appear twice and draws a random sample of 100,000 of the 5,259,281 different names with seed 42. The

CSV file is sorted by name, so the first 100,000 rows would only have covered names that start with a few letters.

If very similar names sit in training and test, the test partly measures what the model has memorized (Lee et al., 2022). The script therefore groups similar names before the split. First it normalizes the names. It replaces & with AND, removes apostrophes and full stops, turns all other punctuation into spaces, and removes a leading THE or WWW and endings like LTD, LIMITED, CO or UK. It then puts names with the same first two remaining words into one group, so ACME HOLDINGS LTD and ACME HOLDINGS (UK) LIMITED stay together, and splits the 95,078 groups about 80/10/10 into 80,142 training, 9,835 validation and 10,023 test names. The rule does not identify real company families. After the normalization, A&Z FARM (LINCOLN) LIMITED and A & L MOTS LTD both start with the words A AND and land in one group. On the other hand, names with the same first word but a different second word can still land in different splits.

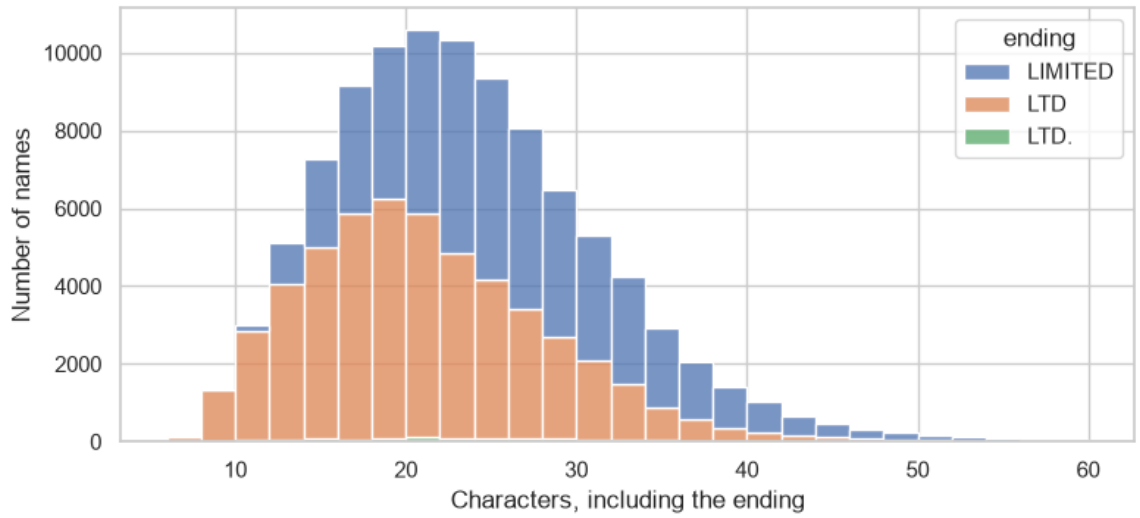
The script also computes a key for every live company. I wrote the key myself. It is based on the "same as" rules in Schedule 3 of the Company, Limited Liability Partnership and Business (Names and Trading Disclosures) Regulations 2015 (SI 2015/17), as Companies House (2026) describes them in its guidance. It uses their lists for legal forms and other endings, punctuation, accented letters, symbols, digits and some words. So HANDS UK LTD and HAND'S LIMITED get the same key. It is still not the official test. My key does not apply every rule in the same order. For example, it gives ACME L.T.D. and ACME LTD different keys, and it cuts a name to 60 characters before it removes the legal form. The notebook shows more examples. I do not know how Companies House applies the rules in detail, and I did not compare my key with its decisions. The key does not cover the exceptions, for example names within the same company

group with written consent. It also does not cover the separate "too like" test, which Companies House decides case by case. The file register_key_hashes.npy holds a 64-bit hash of the key of every company, 5,673,818 different keys for 5,704,711 records. A key match is therefore a possible collision under my key, and a name without a key match can still be the same as or too like a registered name.

The names are short. The median name has 22 characters and 3 words, and Figure 1 shows that most names are between 10 and 40 characters long. 51,620 names end with LTD, 47,376 with LIMITED and 1,004 with LTD. with a full stop. After the preparation 48 different characters remain, capital letters, digits, the space and 11 punctuation marks. The most frequent words are generic business words like SERVICES (in 4.8% of the training names), PROPERTY (2.6%) or HOLDINGS (2.1%). From the samples my impression is that these words are mostly added around a proper name or an invented word, but I did not measure this.

**Figure 1**

*Length of the Company Names in Characters, by Ending*



## Model Design and Training Approach

The model has almost the same structure as the decoder-only Transformer that I built in the course from separate PyTorch classes, but for the project I created it with the Hugging Face Transformers library (Wolf et al., 2020), which is part of my coursework and which I also used in Project 4. A GPT2Config sets the size, and GPT2LMHeadModel creates the model with random weights, so nothing is pretrained or downloaded. A character tokenizer, built from the training names, maps the 48 characters to ids and adds a padding token and an end marker, which stands before and after every name. A token embedding and a learned position embedding turn each character into a vector of 256 numbers. Four Transformer blocks follow, each with causal self-attention over eight heads and a feed-forward part from 256 to 1,024 to 256 units with GELU. As in GPT-2, each part has a layer norm in front and a residual connection around it, and a final layer norm follows the last block (Radford et al., 2019). A linear layer then gives one logit per token, and it shares its weights with the token embedding, which is the main difference to the course model. Smaller differences are the dropout after the embeddings and the mask for the padding token. The causal mask lets every position see only itself and the earlier characters, so the model learns to predict the next character, which is the basis of generation (Vaswani et al., 2017). The model has 3,187,968 parameters. The training loop, the loss and the evaluation are my own PyTorch code. In char_transformer.py I also define the same architecture with my own PyTorch classes. In the notebook I copy the weights of the library model into it and check that both give the same logits up to a difference of about 1e-6, so the full architecture is defined in my own code as well.

The pretrained GPT-2 weights would not have fit this task. GPT-2 learned from WebText, the text of about 8 million web pages (Radford et al., 2019), which contains real

company names, so a copied name could also come from its pretraining. It also splits text into pieces of words with byte pair encoding, while my model works with single characters. Large language models can reproduce parts of their training data word for word, including personal information (Carlini et al., 2021). With my own model there is no unknown pretraining data. I know which 80,142 names the model saw during training and can check whether a generated name is one of them, but an exact match alone does not tell me how the model produced it. I chose this small size myself and did not compare it with other sizes.

The loss is the cross-entropy between the predicted distribution of the next character and the real next character, averaged over all characters and end markers, with the padding ignored. I compute it in my own training loop from the logits of the model, and an attention mask hides the padding. Cross-entropy is the standard loss for models that output a probability distribution (Goodfellow et al., 2016). The optimizer is AdamW, Adam with decoupled weight decay (Loshchilov & Hutter, 2019), with a learning rate of 1e-3, weight decay 0.01 and batches of 128 names. The learning rate and the weight decay are the default values of AdamW in PyTorch, and I did not search for better ones. Dropout of 0.1 after the embeddings, in the attention and after the attention and feed-forward parts switches off random units during training to reduce overfitting (Goodfellow et al., 2016). I trained for 20 epochs and kept the weights of the epoch with the lowest validation loss, the selection idea of early stopping (Goodfellow et al., 2016). All random parts use seed 42.

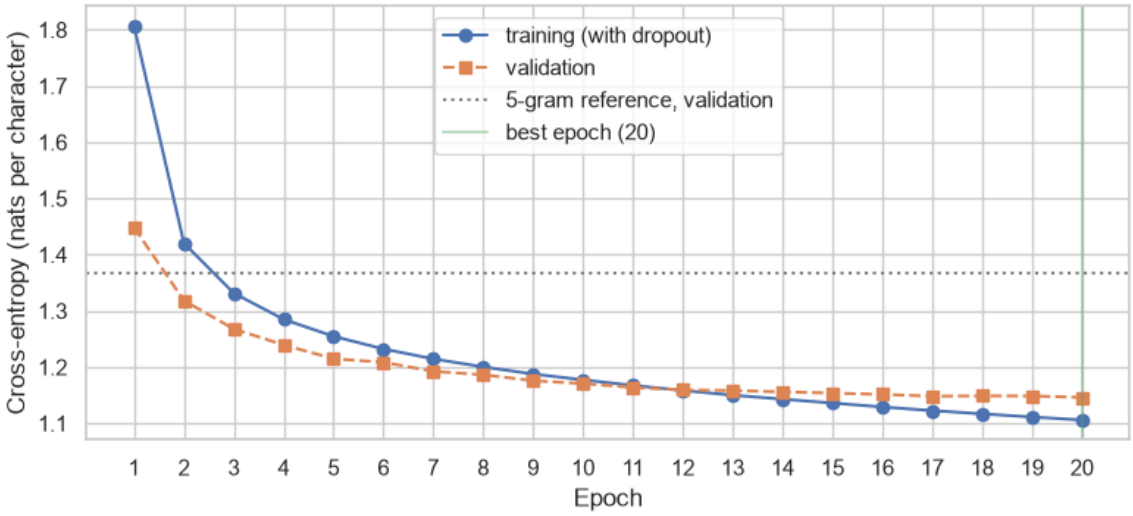
As a reference without deep learning I built a character n-gram model. It counts which character follows the last n - 1 characters and smooths the counts with a small constant k. It does not fall back to shorter contexts. The best of 12 combinations was order 5 with k = 0.1, with a validation loss of 1.3680 nats per character. Figure 2 shows that the Transformer was below this

value from the second epoch (1.3184). Its validation loss reached 1.1707 in epoch 10 and then fell by only 0.0246 more to 1.1461 in epoch 20, while the training loss kept falling to 1.1059. From epoch 12 the training loss was lower than the validation loss, and the gap grew to about 0.04. This gap is not a direct comparison, because I measured the training loss with dropout while the weights still changed during the epoch, and the validation loss with the final weights of the epoch and without dropout. The validation loss did not show a sustained rise within 20 epochs. The lowest validation loss came in the last epoch, whose weights I kept, but the values from epoch 17 to 20 differ by less than 0.003. More epochs might have helped a little, but I did not test this. On the test split the Transformer reached 1.1513 nats per character, a perplexity of 3.16, against 1.3819 for the 5-gram model and 3.8918 for a model that gives the same probability to the 49 tokens other than padding. The 5-gram comparison shows that the Transformer predicts held-out characters better than a model that only uses local contexts, but it does not show which dependencies the Transformer learned.

The notebook uses CUDA if it is available, otherwise the CPU, and I trained the model on the CPU of my Mac. One epoch took about 3.5 minutes, the whole training about 67 minutes and a full run about 69 minutes. On a busy machine an epoch took up to 6 minutes and a full run up to 100 minutes. The notebook trains the model in every full run. Repeated full runs on the CPU gave the same loss in every epoch and the same generated names. All numbers and examples in this report come from my run on the CPU. On other hardware, for example with CUDA, they can differ slightly.

**Figure 2**

*Cross-Entropy per Character During Training*




## Output Evaluation and Interpretation

To generate names I use model.generate of the Hugging Face library with sampling. The model starts with the end marker, samples one character from its predicted distribution, appends it and repeats until it samples the end marker again. A name can have up to 60 characters and then the end marker. The temperature divides the logits before the softmax. A low temperature makes the model choose likely characters more often, a high temperature gives rare characters more chances. Sampling from the full distribution can produce strange choices from its unreliable tail, while choosing only the most likely tokens makes text bland and repetitive (Holtzman et al., 2020). I switched off the top-k filter and the nucleus sampling of the library, so only the temperature changes the distribution, and I compared the temperatures 0.5, 0.8, 1.0 and 1.2 and generated 2,000 names at each, plus 2,000 names from the 5-gram model at temperature 1.0.

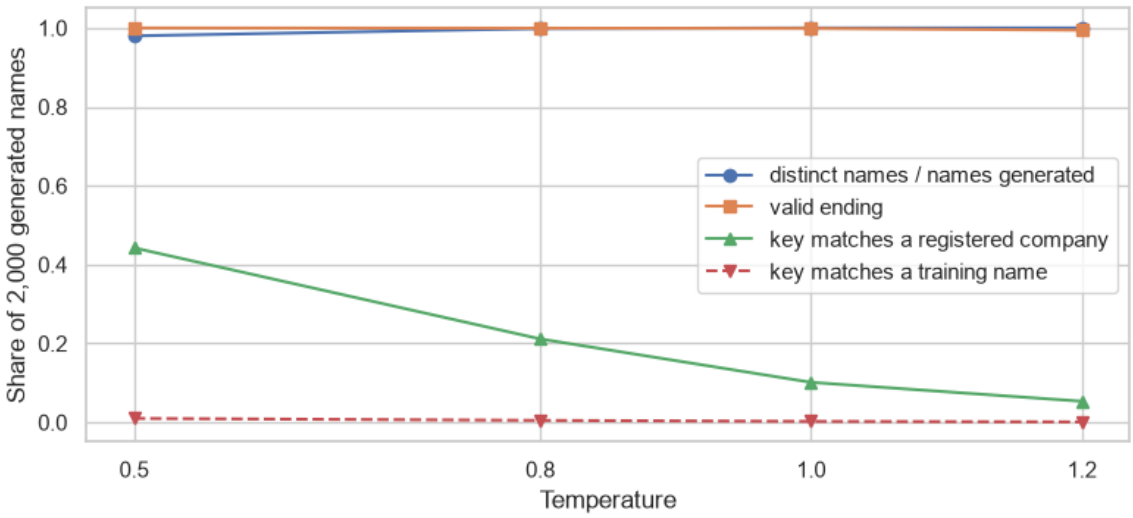
A low loss does not guarantee good samples, because likelihood and sample quality can disagree, and generative models should be judged with checks that fit their use (Theis et al.,

2016). Before I looked at the names I fixed checks for my use case. I counted the finished names, the names with a valid ending, the distinct share (different names divided by names generated) and the exact copies of training names. I also checked whether the key of a name matches the key of a training name or of any company in the snapshot. All 10,023 real test names were found in the register, which tests the lookup but not how well my key follows the official rules.

The Transformer finished 7,993 of its 8,000 names, and 99.45% to 100% of its names had a valid ending. At temperature 0.5 the distinct share, the share of different names, was 98.00%, at the higher temperatures 99.85% to 100%. Exact copies of complete training names were rare in these samples, 16 of 8,000, and 0.10% to 1.00% of the names had the key of a training name. The register check showed a much bigger problem (Figure 3). At temperature 0.5 the key of 44.25% of the names matched a company in the snapshot, at 0.8 21.15%, at 1.0 10.15% and at 1.2 5.35%. These shares include matches with training keys, but without them they are at most 1.00 percentage point lower, for example 9.90% at 1.0. None of 24 sampled matches was an exact copy of a training name or had the key of a training name. Many were short combinations of common words, like THE CARE COMPANY LIMITED and CASTLE DESIGNS LTD, names that look as if they contain a person's name, like HUGHES HOLDINGS LIMITED, or short initials like PMGM LIMITED. At temperature 0.8 names with fewer words matched more often, 47.58% of the names with two words, but only 10.14% of the names with four words. Only 37 of the 1,618 matches had the key of a training name and 16 were exact copies, but my checks do not show whether the model reproduced parts of other names.

**Figure 3**

*Quality Checks and Key Matches by Sampling Temperature*




To see how close the new names are to the training names, I measured for every name the cosine similarity of TF-IDF weighted character 2- to 4-grams (Salton & Buckley, 1988) to its most similar training name. I computed it with scikit-learn, which is part of my machine learning coursework. The check only looks at shared groups of characters. It ignores word order and meaning, so it is not an identity test. 19 pairs reached a similarity of at least 0.999. 17 of them were exact copies, 16 from the Transformer and one from the 5-gram model, and two were ELITE CARE LIMITED, generated at 0.5 and at 0.8, against the training name CARE ELITE LIMITED. For comparison I used a random sample of 2,000 of the 9,835 real validation names, company names not seen during training, with a median similarity of 0.584. This is only a rough reference. The split keeps names with the same first two normalized words out of the training names, which can lower their similarity, but other close variants can still be in the training names. I used the validation names and not the test names, because I chose the temperature with this comparison. In Figure 4 the names at temperature 1.0 have almost the same distribution with a median of 0.575, while the names at 0.5 share more character groups with training names

(median 0.685). At 0.8 the median is 0.622, at 1.2 0.520 and for the 5-gram names only 0.335. Six of the first ten 5-gram names broke into random-looking characters like SIRN'YOP)@6WIR9, because an unseen context of four characters makes every token except padding equally likely under add-k smoothing. Only 32.60% of its names ended correctly. The Transformer produced much more regular names in this comparison, although I did not test how much of this difference comes from attention alone.

**Figure 4**

*Similarity of Each Name to Its Closest Training Name*

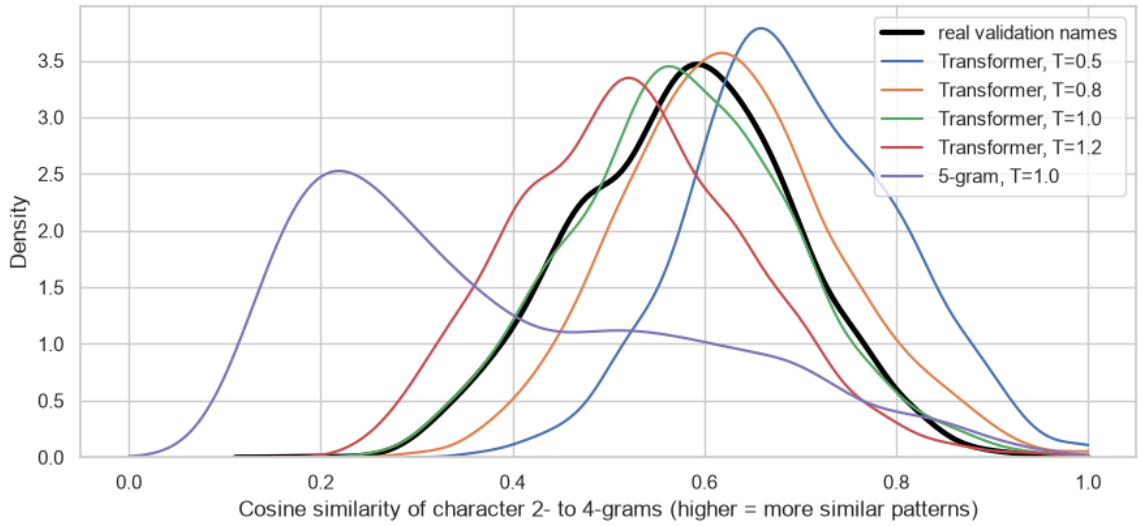
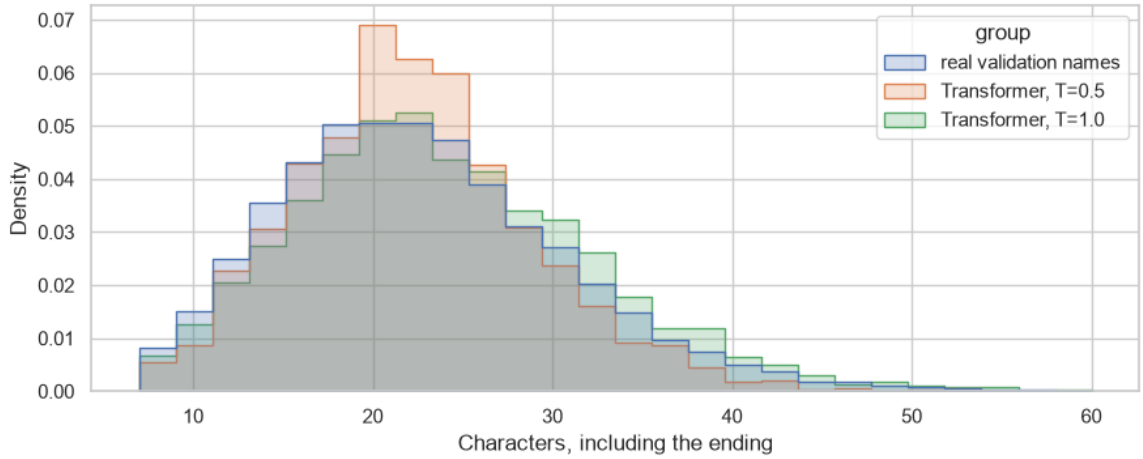



Figure 5 shows that the names at temperature 1.0 follow the real validation names in length up to about 26 characters but include more long names, with a mean of 24.18 characters against 22.95. The names at 0.5 have a mean of 22.50 but are more concentrated between 18 and 26 characters. I also counted the names with at least one word that is not in the training names. For this I split each name on whitespace and left out a final LTD, LTD. or LIMITED. Such a word can be a real name, a number, a code, a misspelling or an invented word, so this shows how new the words are and not how good the names are. For the real validation names the share is 51.10%. The Transformer has 42.70% at 0.8 and 62.85% at 1.0, against 14.15% at 0.5 and

79.60% at 1.2. At 0.5 the model also uses a few words much more often. It used CONSULTING in 10.50% of the names instead of 1.57% in the training data and PROPERTIES in 7.05% instead of 2.18%, while UK fell from 1.79% to 0.25%. This is similar to the observation of Holtzman et al. (2020) that decoding that only picks the most likely tokens leads to bland and repetitive text, although at 0.5 my model still samples from the full distribution.

**Figure 5**

*Length of Generated and Real Names*




Reading the names supports these numbers. I call a name plausible if I can read its words or name parts, if it has a typical structure and the right ending and if its brackets are closed. Of the 2,000 names at temperature 1.0, 1,795 pass my ending, duplicate and register-key checks. They are only candidates for further review, because these filters do not check readability or brackets, and six of them have unbalanced brackets, like HOWLAND ESTATES (26 LIMITED or ARCH CAD (NI) MINEERS) LTD. In a random sample of 15 candidates many look plausible to me, for example ZNM PLUMBING & HEATING SERVICES LIMITED or INDIAN CRICKET KITCHEN LIMITED. Others are weaker. 16228 OPERATIONS RVINE SERVICES LIMITED has a word that looks broken to me, and CAXPY CAPITAL INVESTMENT

LIMITED starts with a word that is hard to read. 14 of the 8,000 names failed my ending check, which only looks at the last word, and 11 of them were generated at 1.2. Seven were cut off at the limit of 60 characters, like ALDERIS BUILDING SOLUTIONS & PROCESS ENGINEERING SERVICES LT, and others misspell the ending, like THE WAIFESAFE LLTD. Two of the five longest names at 1.2 open a bracket that they never close, like WORTH MAN FERRS NORTHERN BUZZAMAJ (CANTRE PROPERTY COACH CHA. At temperature 0.5 the opposite problem appears, CONSTRUCTION LIMITED, ARCHITECTURE LIMITED and ARCHITECTURE LTD each came out 4 times among 2,000 names.

I chose the temperature for the detailed examples with a rule on the validation names. This is an exploratory choice. I wrote it after an earlier version of my notebook had compared the temperatures on the test names, so it was not fixed in advance. The rule takes the temperature whose similarity to the closest training name is distributed most like that of the real validation names, measured with the Kolmogorov-Smirnov distance, the largest gap between two cumulative distributions (Massey, 1951). It picked 1.0 with a distance of 0.0320, against 0.1460 at 0.8, 0.2190 at 1.2 and 0.3820 at 0.5. On the length of the names 0.8 was closer to the validation names (0.0435 against 0.0650). The rule looks at all generated names. After the filters, 1.0 was still closer to the validation names (0.0674) than 0.8 (0.1044), but I added this check only after the rule, so it does not show which temperature is best for the names I would really use. On the test names 1.0 gave a similar distance of 0.0455, but I had already looked at these names before, so this cannot confirm the choice. A similar distance also says nothing about plausibility. 10.15% of the names at 1.0 still match the register, so a register check is needed before any name is used.

## Ethical Considerations and Responsible Use

The first concern comes directly from my results. A generated name can have the same key as a real company even if its key matches no training name. At temperature 1.0 this happened for 9.90% of the names and at 0.5 for 43.25%. These are possible collisions outside the training keys, not proven company identities. In my use case such a name could match a real company in a screening system. If a test or demo output is shared, a real company could appear in a context that has nothing to do with it, for example in a list of suspicious entities. Companies House generally rejects new names that are the same as an existing one (Companies House, 2026), and I used my own key, which is based on these rules, as a filter. The responsible use of this generator therefore includes the register check as a fixed step. For the non-matching test inputs I have in mind, I would exclude every register-key match and review the remaining candidates further. Even then the check is not a guarantee. It covers only the UK register of one month and not the "too like" test. So a name without a match can still be close to a company in the UK, belong to a company in another country or be protected as a trademark. The question of ownership here is less about copyright and more about confusion. A generated name that is close to a real one can create a false link to that company or touch its trademark rights, which the guidance asks applicants to check separately.

The second concern is personal names. Company names can contain personal names, and names in the data like SMITH HOUSING MORPETH LTD look as if they do. Some generated names also look as if they contain a person's name, for example HUGHES HOLDINGS LIMITED and PATRICK CONSULTANCY LIMITED, whose keys match registered companies. I did not verify the people behind these examples, but a generated name that looks like a person's name could still be read as a statement about that person. Language models can

reproduce personal information from their training data (Carlini et al., 2021). In my samples exact copies were rare, 16 of 8,000, but that does not show that the model memorized nothing. For this reason I would not publish generated names as a list without the register check, and I would label them as generated test names that may overlap with real company names. The examples in the notebook are raw outputs, including register-key matches and malformed names. They are not approved test names.

The third concern is misuse. A generator of plausible company names can also help someone to create names for shell companies or to imitate a real company in phishing. The key matches show that the model can produce names that may collide with existing businesses, but I did not test whether people would confuse them. The model produces only names, not documents or websites, and I did not measure whether it makes misuse easier than choosing names by hand. It can produce thousands of names quickly, so I would not offer it as a public name-generation service. The notebook trains the model from scratch in every full run, so anyone with the code and the data can train it again and generate names. In the context of my capstone it is meant only to create test inputs for a matching system that is supposed to detect exactly this kind of name. There is also a bias. I removed names with characters outside the allowed set (6,984), among them names with accented letters, and names with other endings (7,971), for example the Welsh form CYF. The training names also reflect which companies exist in the UK. A matching system tested only with such names would be tested less on names from other languages, and I expect that matching errors matter there in compliance checks, although I did not test this.

## Limitations and Future Improvements

The model and its evaluation have clear limits. My register key does not follow the official test in every detail, and I did not measure how often it merges or separates names differently from Companies House. My similarity measure only compares character groups and says nothing about meaning or word order. I trained one configuration and always used seed 42, so I did not test how much the results change with another seed or another size of the model. All results come from the CPU of my Mac, and I could not test CUDA. I judged the quality of the names by reading samples myself, which is my own judgment and not a tested measure of plausibility. The test loss and the name-quality comparisons cover the selected private limited companies of one snapshot. The register lookup covers all live companies in that snapshot, but no other month or country.

Realistic next steps follow from these limits. First, the register check could add a fuzzy match for names that are only too similar, which is the same kind of matching the due diligence agent will need. Second, the evaluation could add a small blind test where several people try to tell real and generated names apart, and a rule for choosing the temperature on the filtered names could be fixed in advance and checked on names I have not looked at before. Third, the model could learn the legal form as a condition, so it can write names for other company types, and the excluded names with accents could be added back to reduce the language bias. Finally, a real benchmark for entity matching needs pairs with a known answer. Controlled variants of real company names, for example with changed punctuation or word order, could give such pairs as a separate test set, while the generated names would stay candidates for inputs that should not match anything. I have not shown that they are true non-matches.

## References

Carlini, N., Tramèr, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K., Roberts, A., Brown, T., Song, D., Erlingsson, Ú., Oprea, A., & Raffel, C. (2021). Extracting training data from large language models. In *Proceedings of the 30th USENIX Security Symposium* (pp. 2633–2650). USENIX Association. https://www.usenix.org/conference/usenixsecurity21/presentation/carlini-extracting

Companies House. (n.d.). *Free Company Data Product*. Retrieved October 16, 2026, from https://download.companieshouse.gov.uk/en_output.html

Companies House. (2019). *Our public task, copyright and Crown copyright*. GOV.UK. https://www.gov.uk/government/publications/companies-house-accreditation-to-information-fair-traders-scheme/public-task-copyright-and-crown-copyright

Companies House. (2026). *Incorporation and names*. GOV.UK. https://www.gov.uk/government/publications/incorporation-and-names/incorporation-and-names

Company, Limited Liability Partnership and Business (Names and Trading Disclosures) Regulations 2015, SI 2015/17, sch. 3. https://www.legislation.gov.uk/uksi/2015/17/schedule/3/made

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org

Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. In *International Conference on Learning Representations*. https://openreview.net/forum?id=rygGQyrFvH

Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., Callison-Burch, C., & Carlini, N. (2022). Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 8424–8445). Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.acl-long.577

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations*. https://openreview.net/forum?id=Bkg6RiCqY7

Massey, F. J., Jr. (1951). The Kolmogorov-Smirnov test for goodness of fit. *Journal of the American Statistical Association, 46*(253), 68–78. https://doi.org/10.1080/01621459.1951.10500769

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners*. OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

Theis, L., van den Oord, A., & Bethge, M. (2016). A note on the evaluation of generative models. In *International Conference on Learning Representations*. https://arxiv.org/abs/1511.01844

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30*. Curran Associates.

https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., . . . Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 38–45). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-demos.6